Question

In this project we will be working with a fake advertising data set, indicating whether or not a particular internet user clicked on an Advertisement on a company website. We will try to create a model that will predict whether or not they will click on an ad based off the features of that user.

This data set contains the following features:

- 'Daily Time Spent on Site': consumer time on site in minutes
- 'Age': cutomer age in years
- 'Area Income': Avg. Income of geographical area of consumer
- 'Daily Internet Usage': Avg. minutes a day consumer is on the internet
- 'Ad Topic Line': Headline of the advertisement
- 'City': City of consumer
- 'Male': Whether or not consumer was male
- 'Country': Country of consumer
- 'Timestamp': Time at which consumer clicked on Ad or closed window
- 'Clicked on Ad': 0 or 1 indicated clicking on Ad

Understanding Logistic Regression 📊

To solve this project, we first need to understand the core algorithm at work: **Logistic Regression**.

At its technical core, Logistic Regression is a foundational machine learning technique used to predict a *categorical* value. While Linear Regression predicts a continuous number (like estimating the price of a house), Logistic Regression predicts a binary outcome—usually a simple **Yes/No**, **True/False**, or **1/0**. It does this by fitting an S-shaped curve (a logistic function) to the data, figuring out the best mathematical boundary to divide information into distinct categories with the lowest possible error.

In Simple Terms: The "Ad Click" Scenario 🖱️

In logistic regression, we use an algorithm to teach a computer how to recognize patterns in behavior. We want the machine to look at a user's profile and answer one specific question: *"Will this person click on our advertisement?"*

To teach the computer, we follow a simple, step-by-step workflow:

- **Step 1: Splitting the Data (The 80/20 Rule)** 🍕 Imagine we have the data of 100 website visitors. We don't give the machine all 100 records at once. Instead, we divide the data into two piles. We use **80%** of the data (80 users) as a textbook for the machine to study, and we hide the remaining **20%** (20 users) to use later as a final exam.
- **Step 2: Training the Model (Studying)** 🧠 During training, we give the model the 80 users' inputs (like their 'Age', 'Area Income', and 'Daily Time Spent on Site') *along with* the actual answers (whether they actually clicked the ad or not). By looking at these inputs and outputs together, the model figures out the hidden rules. It might learn, for example, that younger users with high daily internet usage are highly likely to click.
- **Step 3: Testing the Model (The Pop Quiz)** 📝 Now, we need to test if the model actually learned the concepts or just memorized the textbook. We give the model the inputs for the remaining 20 users, but we **hide the actual answers**. The model must calculate the probability and make a firm prediction: *Click (1)* or *No Click (0)*.
- **Step 4: Evaluating the Performance (Grading)** 🎯 Because we secretly kept the true answers for those 20 users, we can compare the machine's predictions against reality. Instead of checking if a plotted point is close to a line (like in Linear Regression), we check for direct matches. Did it predict a click when the user actually clicked? The more correct matches it gets, the higher its **Accuracy**.

Bringing It Back to This Project 💻

In this specific project, we use this model entirely for prediction. We are feeding the model specific details from our fake advertising dataset—such as consumer age, area income, and daily internet usage. Based on those features, the model predicts a **0** (will ignore the ad) or a **1** (will click the ad).

So, whenever you think of Logistic Regression, just remember this simple path: **Give the model data ➡️ Train the model ➡️ Test the model ➡️ Evaluate its accuracy ➡️ Use it to predict categories!** 🚀

Answer Code

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

ad_data = pd.read_csv('advertising.csv')

print('___'*500)
print(ad_data.head())
print('___'*500)
print(ad_data.info())
print('___'*500)
```

Understanding the Code: Step-by-Step 🧑‍🏫

Let's break down exactly what our Python script is doing. We already know the first few lines are just setting up our environment and expanding our pandas table so it doesn't get cut off. Here is what happens next:

- **Line 10 (Reading the Data):** We use `pd.read_csv()` to load our advertising dataset into Python so we can work with it.
- **Line 13 (Checking the Head):** We print `ad_data.head()`. This is absolutely necessary to get a visual idea of what our data actually looks like. It shows us the first 5 rows so we can understand the columns we are dealing with.
- **Line 15 (Getting Data Info):** We print `ad_data.info()`. **Pay close attention to this step!** Machine learning models are essentially math engines—they *only* understand numerical data (like integers and floats). They cannot read raw text (strings). By checking the info, we can easily figure out which columns are numbers and which are text. This tells us exactly which data we can include in our model and which data we have to leave out.

Setting Up Our Variables (Lines 23-27) 🎯

```
22
23  from sklearn.model_selection import train_test_split
24  X = ad_data[['Daily Time Spent on Site', 'Age', 'Area Income','Daily Internet Usage', 'Male']]
25  y = ad_data['Clicked on Ad']
26
27  X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.33, random_state=42)
28
```

Just like we did in the Linear Regression project, we need to choose our `X` dataset (the inputs) and our `y` dataset (the output).

- What are we trying to predict? We want to know whether a person will click on the ad or not!
- Because "Clicked on Ad" is our final prediction, it goes strictly into our **`y` variable**.
- We then grab all the useful numerical columns we found earlier (like 'Age', 'Area Income', and 'Daily Internet Usage') and put them into our **`X` variable** to act as the clues.

Splitting the Data (Line 27) 🍕 As usual, we use `train_test_split` to chop our data into two parts: a training set to teach the model, and a testing set to quiz it later.

Bringing in the Model (Line 29) 🧠 This is where things change! If you remember our previous project, we imported `LinearRegression`. But since we are predicting a category here (Click vs. No Click), we import the **`LogisticRegression`** model instead.

```
28
29  from sklearn.linear_model import LogisticRegression
30
31  logmodel = LogisticRegression(max_iter=1000)
32  logmodel.fit(X_train,y_train)
33
34  predictions = logmodel.predict(X_test)
35
36  from sklearn.metrics import classification_report
```

Training the Model (Lines 31 - 32) 📚

- `logmodel = LogisticRegression()`: First, we officially create the "brain" of our model and name it `logmodel`.
- `logmodel.fit(X_train, y_train)`: Next, we tell the model to study! The `.fit()` command is the actual training process. We hand the model our training

inputs (`X_train`) along with the correct answers (`y_train`) and say, *"Here is the data, figure out the hidden patterns!"*

Testing and Grading the Model (Line 40) 🎓 After we ask the model to make its predictions on the hidden test data, we need to see how well it did. In Line 40, we print a **classification report**. This is basically the model's final report card! It compares the model's predicted answers against the actual real-world answers to measure the overall **accuracy** of our model.

This is the error table that we received by executing line 40

```
================================================ ACCURACY TEST ==========================
              precision    recall  f1-score   support

           0       0.96      0.98      0.97       162
           1       0.98      0.96      0.97       168

    accuracy                           0.97       330
   macro avg       0.97      0.97      0.97       330
weighted avg       0.97      0.97      0.97       330


Process finished with exit code 0
```

📊 Interpreting the Classification Report

The table you are looking at is the Classification Report. Because Logistic Regression predicts distinct categories (like Yes or No) rather than continuous numbers, we don't measure numerical distance like we did in Linear Regression (no MAE or RMSE here). Instead, we measure how often the model guessed the correct category.

Understanding the Rows (0 and 1)

- 0 represents the users who did *not* click on the ad.
- 1 represents the users who *did* click on the ad.

1. Support (The Headcount) Support simply tells us how many actual people were in each category during this test.

- Your Support is 162 for class 0, and 168 for class 1.
- This means out of the 330 total people the model was tested on, 162 actually ignored the ad, and 168 actually clicked it.

2. Precision (The "Quality" of Guesses) Precision answers the question: *When the model predicts a user will click, how often is it actually right?*

- Your Precision for class 1 (Clicks) is 0.98 (or 98%).
- This means that out of all the people the model *claimed* would click the ad, it was correct 98% of the time. It rarely falsely accuses someone of clicking when they didn't. A higher precision means fewer false alarms.

3. Recall (The "Detection" Rate) Recall answers a different question: *Out of all the people who ACTUALLY clicked the ad, how many did our model successfully find?*

- Your Recall for class 1 (Clicks) is 0.96 (or 96%).
- This means the model successfully identified 96% of the real clickers, but it missed about 4% of them. A higher recall means you are successfully catching almost everyone who matters.

4. F1-Score (The Balance) F1-score is the harmonious average between Precision and Recall. It is useful when you want a single metric to tell you if the model is well-balanced.

- Your F1-Score is 0.97 for both classes.
- Because 0.97 is extremely close to a perfect 1.0, this tells us your model isn't sacrificing Precision to get a better Recall (or vice versa). It is highly balanced and performs brilliantly.

5. Accuracy (The Big Picture) Accuracy is the most straightforward metric: *Overall, what percentage of its total predictions were dead-on correct?*

- Your Accuracy is 0.97 (or 97%).
- Out of all 330 test users, the model guessed their behavior correctly 97% of the time.

✅ With an incredibly high accuracy of 97% and perfectly balanced F1-scores of 0.97 across the board, this Logistic Regression model is performing phenomenally well. You can confidently trust this model to analyze the data and accurately distinguish between users who will click your advertisement and users who will ignore it.

Deciding the answer

As I mentioned earlier, our early linear regression model, if you remember, we use it to analyse to get our answer, We analyse the co efficient to decide where we should rely on website or mobile phone, If you remember I told that we can use this models to get answer to our questions by several ways. One way is to analyse, but in this question we use a prediction method. It depends on question to question and engineers basically should be able to determine what method we should use. **Now let's review the final part of our answer code.**

```
# ============================================================================================
print('\n' + '==='*30)
print("🔮 Let's Predict a New User's Behavior!")
print('==='*30)

# Step 1: Collect data from the user via the console
time_spent = float(input("Enter Daily Time Spent on Site in minutes (e.g., 65.5): "))
age = int(input("Enter User's Age (e.g., 30): "))
income = float(input("Enter Area Income (e.g., 60000.00): "))
internet_usage = float(input("Enter Daily Internet Usage in minutes (e.g., 200.5): "))
is_male = int(input("Is the user Male? (Type 1 for Yes, 0 for No): "))

# Step 2: Store the collected data in a pandas DataFrame
# Why a DataFrame? So the column names match exactly what the model studied during training!
new_user_data = pd.DataFrame({
    'Daily Time Spent on Site': [time_spent],
    'Age': [age],
    'Area Income': [income],
    'Daily Internet Usage': [internet_usage],
    'Male': [is_male]
})

# Step 3: Feed the new data to our trained 'logmodel' for a prediction
new_prediction = logmodel.predict(new_user_data)

# Step 4: Display the result cleanly
print('\n' + '==='*30)
if new_prediction[0] == 1:
    print("🎯 PREDICTION: This user WILL likely CLICK on the ad! (Output: 1)")
else:
    print("🛑 PREDICTION: This user will likely IGNORE the ad. (Output: 0)")
print('==='*30)
```

What is happening here? 🤷

- `input()`: This function pauses the script and waits for you to type something into the terminal. We wrap it in `float()` or `int()` to ensure the text you type is converted into the strict mathematical numbers the model requires.
- Creating the DataFrame: If we just handed the model a raw list of 5 numbers, Scikit-Learn would throw a warning because it wouldn't know which number belongs to which category. By putting our single user's data into a mini DataFrame, we give the model the exact labels it expects.
- `logmodel.predict()`: This is the magic step. The model looks at the 5 new numbers, applies the S-curve math it learned during training, and outputs an array containing a single number: `[1]` or `[0]`.
- `new_prediction[0]`: Because the prediction comes back wrapped in an array (like a list), we use `[0]` to grab that very first specific number and use an `if/else` statement to print a human-readable result.

```
================================================================================
🔮 Let's Predict a New User's Behavior!
================================================================================
Enter Daily Time Spent on Site in minutes (e.g., 65.5): 55
Enter User's Age (e.g., 30): 25
Enter Area Income (e.g., 60000.00): 55000
Enter Daily Internet Usage in minutes (e.g., 200.5): 40
Is the user Male? (Type 1 for Yes, 0 for No): 1

================================================================================
🎯 PREDICTION: This user WILL likely CLICK on the ad! (Output: 1)
================================================================================
```